



N	number of payments / deposits
I	fixed periodic interest rate
FV	future value of the loan / annuity
PV	present value of the loan / annuity
P	fixed periodic payment / deposit amount

1 Background

What are the mathematics behind the financial calculations of the HP 12c financial calculator or Google Sheets financial formulas?

The goal of this exercise is to implement an MIT App Inventor app to mimic the behavior of the HP 12c. This calculator has many financial calculations available, but the basic operation includes five buttons in the top row to calculate n , i , PV , PMT , or FV , given the values of the other four¹.

2 Equations

The basic loan and annuity calculations with regular fixed periodic payments / deposits and a fixed periodic interest rate involve the following variables:

¹These correspond to the Google Sheets functions:

- ▶ `NPER(rate, payment_amount, present_value, [future_value], [end_or_beginning])`
- ▶ `RATE(number_of_periods, payment_per_period, present_value, [future_value], [end_or_beginning], [rate_guess])`
- ▶ `PV(rate, number_of_periods, payment_amount, [future_value], [end_or_beginning])`
- ▶ `PMT(rate, number_of_periods, present_value, [future_value], [end_or_beginning])`
- ▶ `FV(rate, number_of_periods, payment_amount, [present_value], [end_or_beginning])`

²Annuities with end-of-period deposits are called *ordinary annuities* (also ‘annuity immediate’) marked as `END`; annuities with beginning-of-period deposits are called ‘annuity due’ marked as `BGN`. The formulas for beginning-of-period annuities must include an additional factor of $(1 + I)$. For beginning-of-period payments, substitute $P' = P(1 + I)$ for P in (2.1).

Dereivation of the **Time Value of Money** formula (2.1) shows the relationship between the three terms: the future value FV , the exponential growth of PV over N periods and the adjusted exponential growth for each periodic deposit P over N periods — deposited at the end² of each period, when the interest is also applied.

$$0 = \underbrace{FV}_{\text{Future Value}} + \underbrace{[PV(1+I)^N]}_{\text{Present Value exponential growth}} \quad (2.1)$$

$$+ \underbrace{[P + P(1+I)^1 + P(1+I)^2 + \cdots + P(1+I)^{N-1}]}_{\text{Periodic payment exponential growth over } N \text{ periods}}$$

$$= FV + [PV(1+I)^N] \quad (2.2)$$

$$+ P[(1+I)^0 + (1+I)^1 + (1+I)^2 + \cdots + (1+I)^{N-1}]$$

$$= FV + [PV(1+I)^N] + P \left[\sum_{k=0}^{N-1} (1+I)^k \right] \quad (2.3)$$

The third term in (2.3) is a *geometric series* whose sum S_n is given by (2.4).

$$S_n \equiv \sum_{k=0}^n r^k = \frac{(1 - r^{n+1})}{(1 - r)} = \frac{(r^{n+1} - 1)}{(r - 1)} \quad (2.4)$$

Substituting $r = (1 + I)$ and $n = (N - 1)$ into (2.4) for the right-most bracketed expression of (2.7) yields:

$$0 = FV + [PV(1 + I)^N] + P \left[\sum_{k=0}^{N-1} (1 + I)^k \right] \quad (2.5)$$

$$0 = FV + [PV(1 + I)^N] + P \left[\frac{((1 + I)^{(N-1)+1} - 1)}{((1 + I) - 1)} \right] \quad (2.6)$$

$$0 = FV + [PV(1 + I)^N] + P \left[\frac{(1 + I)^N - 1}{I} \right] \quad (2.7)$$

The **Time Value of Money** formula (2.7) is a balanced financial transaction. The sum of the present values of all cash flows must balance and equal zero. In (2.7) $I > 0$ and $N > 0$, but FV , PV and P can be positive or negative. That implies a *cash flow sign convention*: think of it as your pocket being the center of the universe. If money leaves your pocket (investments, payments), it is negative. If money enters your pocket (loans received, withdrawals), it is positive.

2.1 Future value

Solving (2.7) for FV yields (2.1.1).

$$FV = - \left[[PV(1 + I)^N] + P \left[\frac{(1 + I)^N - 1}{I} \right] \right] \quad (2.1.1)$$

2.2 Present value

Solving (2.7) for PV yields (2.2.3).

$$PV = - \left[\frac{FV + P \left[\frac{(1 + I)^N - 1}{I} \right]}{(1 + I)^N} \right] \quad (2.2.1)$$

$$= - \left[\frac{FV}{(1 + I)^N} + \frac{P}{(1 + I)^N} \left[\frac{(1 + I)^N - 1}{I} \right] \right] \quad (2.2.2)$$

$$PV = - \left[\frac{FV}{(1 + I)^N} + P \left[\frac{1 - (1 + I)^{-N}}{I} \right] \right] \quad (2.2.3)$$

2.3 Payment

Solving (2.1.1) for P yields (2.3.1).

$$P = - \left[\frac{FV + [PV(1 + I)^N]}{\left[\frac{(1 + I)^N - 1}{I} \right]} \right] \quad (2.3.1)$$

Solving (2.2.3) for P yields (2.3.2).

$$P = - \left[\frac{[FV(1 + I)^{-N}] + PV}{\left[\frac{1 - (1 + I)^{-N}}{I} \right]} \right] \quad (2.3.2)$$

2.4 Number of periods

Solving (2.7) for N yields (2.4.7).

$$0 = FV + PV(1+I)^N + \frac{P(1+I)^N}{I} - \frac{P}{I} \quad (2.4.1)$$

$$\frac{P}{I} - FV = \frac{P(1+I)^N}{I} + PV(1+I)^N \quad (2.4.2)$$

$$P - FV \times I = P(1+I)^N + PV \times I(1+I)^N \quad (2.4.3)$$

$$P - FV \times I = [P + PV \times I](1+I)^N \quad (2.4.4)$$

$$\frac{P - FV \times I}{P + PV \times I} = (1+I)^N \quad (2.4.5)$$

$$\ln \left[\frac{P - FV \times I}{P + PV \times I} \right] = \ln[(1+I)^N] = N \ln[1+I] \quad (2.4.6)$$

$$N = \frac{\ln \left[\frac{P - FV \times I}{P + PV \times I} \right]}{\ln[1+I]} \quad (2.4.7)$$

2.5 Interest rate

There is no closed-form algebraic solution for I . It can be calculated using the Newton-Raphson method to find the root of the **Time Value of Money** formula (2.7). Another approach — since we will ultimately implement this in a MIT App Inventor app to mimic the behavior of the HP 12c and don't necessarily want to include lessons on the Newton-Raphson method — is to exhaustively test interest rates to an order of magnitude M to find one closest to the root of (2.7). See section 3.1.

³interest ACCumulation factor

⁴Future Value Interest Factor of Annuity

⁵Present Value Interest Factor of Annuity

2.6 Simplifying functions

Other simplifying functions that encapsulate common financial vocabulary ($ACC(I,N)$ ³, $FVIFA(I,N)$ ⁴, and $PVIFA(I,N)$ ⁵) are found as multipliers in (2.7), (2.1.1), and (2.2.3).

$$ACC(I,N) = (1+I)^N \quad (2.6.1)$$

$$FVIFA(I,N) = \frac{(1+I)^N - 1}{I} \quad (2.6.2)$$

$$FVIFA(I,N) = \frac{ACC(I,N) - 1}{I} \quad (2.6.3)$$

$$= PVIFA(I,N) \times ACC(I,N) \quad (2.6.4)$$

$$PVIFA(I,N) = \frac{1 - (1+I)^{-N}}{I} \quad (2.6.5)$$

$$= \frac{(1+I)^N (1 - (1+I)^{-N})}{(1+I)^N (I)} \quad (2.6.6)$$

$$= \frac{(1+I)^N - 1}{(1+I)^N (I)} \quad (2.6.7)$$

$$PVIFA(I,N) = \frac{ACC(I,N) - 1}{ACC(I,N)(I)} \quad (2.6.8)$$

$$= FVIFA(I,N) \div ACC(I,N) \quad (2.6.9)$$

2.6.1 FVIFA

Breaking down the components for $FVIFA = \frac{(1+I)^N - 1}{I}$:

► $(1+I)^N$ (The Growth Engine): This represents

how much \$1 would grow if left alone for N periods.

- -1 (The Principal Removal): We subtract 1 to isolate the "growth" or "spread" created by the series of payments rather than just a single lump sum.
- $\div I$ (The Accumulation Scaling): Dividing by the rate converts that growth into the total sum of the periodic stream.

Real-world use: "If I save \$100 a month for 10 years at 5%, what is my total balance?" $FV = P \times FVIFA$

2.6.2 PVIFA

Breaking down the components for the $PVIFA$:

- $(1 + I)^{-N}$ (The Discount Factor): This tells you the value of \$1 received N periods in the future. Because the exponent is negative, the value is always less than \$1.
- $1 - (\dots)$ (The Inverse Relationship): This calculates the "missing" interest — the difference between the total face value of the payments and their current discounted value.
- $\div I$ (The Present Value Scaling): This scales that difference into the total "lump sum" equivalent you would need today to generate those future payments.

Real-world use: "If I pay \$2,000 a month for 30 years at 6%, how much house can I buy today?"
 $PV = P \times PVIFA$

Rewriting (2.1.1), (2.2.3), (2.3.1), and (2.3.2) in terms of these simplifying functions yields:

$$FV = -[PV \times ACC(I, N) + P \times FVIFA(I, N)] \quad (2.6.10)$$

$$PV = -[FV \div ACC(I, N) + P \times PVIFA(I, N)] \quad (2.6.11)$$

$$PMT = -\left[\frac{FV + PV \times ACC(I, N)}{FVIFA(I, N)} \right] \quad (2.6.12)$$

$$= -\left[\frac{FV \div ACC(I, N) + PV}{PVIFA(I, N)} \right] \quad (2.6.13)$$

3 Scenarios

These are common financial scenarios under the assumptions that $I > 0$ and $N > 0$. Also, to maintain these assumptions when *calculating* I or N , it must also be the case that PV , P , and FV cannot all have the same sign.

Scenario	PV	P	FV	Logical Interpretation
'Black Hole'	≤ 0	< 0	< 0	Error: Money is paid at every stage while remaining in debt.
Savings Goal	≤ 0	< 0	> 0	Invested a lump sum (or zero); make monthly deposits; withdraw a lump sum later.
Payout Annuity	< 0	> 0	≤ 0	Invested a lump sum; receive a regular "salary" back; to a residual debt (or zero).
Endowed Annuity	< 0	> 0	≥ 0	Invested a lump sum; receive a regular "salary" back; to a residual balance (or zero).
Standard Loan	> 0	< 0	≤ 0	Received a lump sum; making monthly payments to a balloon payment (or zero).
Incented Lease	≥ 0	< 0	> 0	Received a lump sum (or 0); paid rent; received a deposit back.
Reverse Mortgage	≥ 0	> 0	< 0	Received a lump sum (or 0) AND monthly checks; balance owed later.
'Money Tree'	> 0	> 0	> 0	Error: Money is received at every stage without cost.

3.1 Sample implementation

This is a sample Python implementation of these financial formulas using end-of-period ('annuity immediate') calculations. Either of the `calcPMT` functions can be used as they are equivalent. The `calcI` function iterates over all possible values in order-of-magnitude `m` on $(0, 1]$ and picks the best one.

NOTE

TODO

- Describe under what circumstances I & N calculations fail.
- Add calculations for *total interest* (CUMIPMT) and *total principle* (CUMPRINC) — and other useful calculations (ACC, FVIFA, PVIFA)?

```

1 123456789012345678901234567890123456789012345678901234567890123456789012345678901234567890
2 # Sample calculations for time value of money (TVM)
3 acc = lambda i, n: (1 + i) ** n
4 FVIFA = lambda i, n: (acc(i, n) - 1) / i # PVIFA(i, n) * acc(i, n)
5 PVIFA = lambda i, n: (1 - 1 / acc(i, n)) / i # FVIFA(i, n) / acc(i, n)
6
7 calcN = lambda i, pv, pmt, fv: \
8     math.log((pmt - fv * i) / (pmt + i * pv)) / math.log(1 + i)
9 calcPV = lambda n, i, pmt, fv: -fv / acc(i, n) - pmt * PVIFA(i, n)
10 calcPMT = lambda n, i, pv, fv: (-fv / acc(i, n) - pv) / PVIFA(i, n)
11 calcPMT = lambda n, i, pv, fv: (-fv - pv * acc(i, n)) / FVIFA(i, n)
12 calcFV = lambda n, i, pv, pmt: -pv * acc(i, n) - pmt * FVIFA(i, n)
13
14 def calcI(n, pmt, pv, fv, m=5):
15     """Return value of i = step / 10 ** m for which calcFV(n, i, pv, pmt)
16     is closest to fv. Check all steps decimal values of i.
17     This approach uses a dictionary."""
18     candidates = \
19         { abs(fv - calcFV(n, step / 10 ** m, pv, pmt)): step / 10 ** m
20           for step in range(-(10 ** m) - 1, 10 ** m + 1) if step != 0 }
21     return candidates[min(candidates)]
22
23 def calcI(n, pmt, pv, fv, m=100000):
24     """Return value of i = step / m for which calcFV(n, i, pv, pmt) is
25     closest to fv. Check all 2 x steps decimal values of i on [-m, +m].
26     This approach uses iteration."""
27     minI, minDiff = math.inf, math.inf
28     for step in range(-m - 1, +m + 1):
29         if step != 0:
30             i, diff = step / m, abs(fv - calcFV(n, step / m, pv, pmt))
31             if diff < minDiff:
32                 minI, minDiff = step / m, diff
33     return minI

```